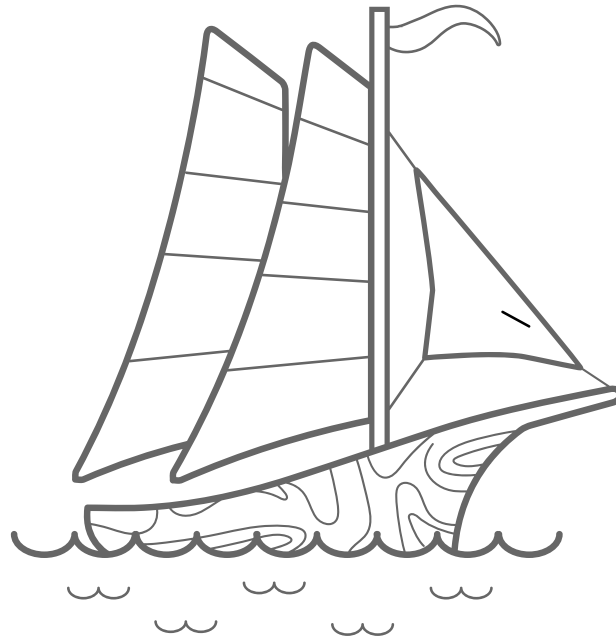


LangQuest

Guide utilisateur

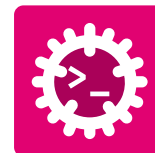


LangQuest

HES-SO Valais//Wallis
Systems Engineering • Infotronics
Advanced Programming • Silvan Zahno
Fall Semester 2026 • I3

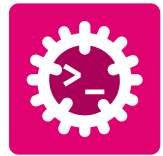
Silvan Zahno, Axel Amand

05.08.2026 • v1.0.0 • final



Contenu

1	Introduction	3
2	Installation	4
2.1	Toolchain Rust	4
2.1.1	Installation	4
2.1.2	Installer les extensions cargo	5
2.1.3	Vérification	6
2.2	LangQuest	7
2.2.1	Installation	7
2.2.2	Utilisation rapide	7
2.2.3	Associer les fichiers à l'éditeur	8
2.3	Éditeur de code Zed	9
2.3.1	Installation	9
2.3.2	Vérification	9
2.4	Just - Exécuteur de commandes	10
2.4.1	Installation	10
2.4.2	Vérification	10
3	Guide LangQuest	11
3.1	Lancer LangQuest	11
3.2	Page d'ensemble	12
3.2.1	Résumé des raccourcis	13
3.3	Page des exercices	14
3.3.1	Théorie	14
3.3.2	Tâches	14
3.3.3	Débogage	14
3.3.4	Résultats	15
3.3.5	Indices et solution	15
3.3.6	Résumé des raccourcis	16
3.4	Page à propos	17
3.5	Dépannage	17
	Glossary	19



1 Introduction

LangQuest tel que présenté sur sa page GitHub :



LangQuest

A terminal-based, interactive programming exercise runner. Inspired by Rustlings and 100 Exercises to Learn Rust, LangQuest extends the concept to multiple languages - work through hands-on exercises in Rust, Go, C++, Python, RISC-V assembly, and Markdown with real-time feedback, progress tracking, and a built-in hint system.

Cet outil est développé par [Zahno Silvan](#) et publié sous licence MIT.

Le code source est disponible sur <https://github.com/tschinz/langquest>.

LangQuest est un outil interactif dont l'interface tourne en lignes de commandes grâce à [Ratatui](#) et permet de travailler sur des exercices pratiques en Rust, Go, C++, Python, RISC-V assembly et Markdown avec un retour en temps réel, un suivi de progression et un système d'indices intégré.

Le document suivant fournit un guide utilisateur complet de LangQuest couvrant l'installation, l'utilisation et le dépannage pour vous aider à tirer le meilleur parti de cet outil d'apprentissage interactif.



2 | Installation

Cette section concerne tous les outils liés à LangQuest. Suivez les sous-sections dans l'ordre - certains outils dépendent d'installations précédentes.

2.1 Toolchain Rust



Figure 1 - Rust Programming Language

Plusieurs outils s'installent via **Cargo**, le gestionnaire de paquets Rust.

Rust est installé et géré via **rustup**, l'installateur officiel de la toolchain Rust. **rustup** installe le compilateur (**rustc**), l'outil de build et gestionnaire de paquets (**cargo**), ainsi que des composants et cibles additionnels.

2.1.1 Installation

Consultez rust-lang.org/tools/install pour les instructions complètes.

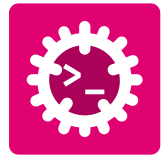
Ouvrez un terminal et exécutez :

```
1 curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```



Suivez les étapes affichées (l'installation par défaut est recommandée). Après l'installation, réouvrez votre terminal ou rechargez l'environnement :

```
1 source "$HOME/.cargo/env"
```



Téléchargez et exécutez **rustup-init.exe** depuis <https://win.rustup.rs/>.

Si les outils **MSVC** ne sont pas installés, acceptez leur installation lorsqu'elle est proposée :



Continuez ensuite l'installation (l'installation standard est recommandée) :

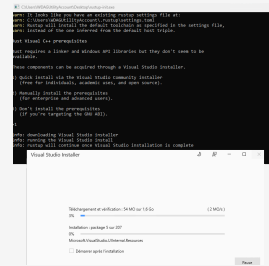


Figure 2 - MSVC installation through rustup

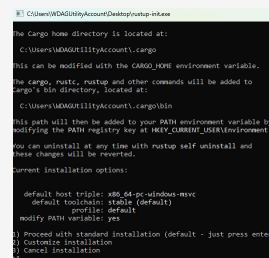


Figure 3 - Rust installation through rustup on Windows

2.1.2 Installer les extensions cargo

Dans un nouveau terminal, exécutez les commandes suivantes pour installer des extensions **cargo** utiles pour le formatage et l'analyse statique :

```
1 # Install rustfmt for code formatting
2 rustup component add rustfmt
3 # Install clippy for linting and code analysis
4 rustup component add clippy
```



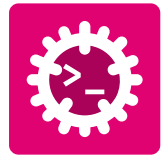
2.1.3 Vérification

Vérifiez que les commandes suivantes affichent un numéro de version sans erreur :



```
1 rustup --version
2 rustc --version
3 cargo --version
4 cargo fmt --version
5 cargo clippy --version
```

Toutes les commandes doivent afficher une version. En cas d'échec, assurez-vous que **\$HOME/.cargo/bin** (macOS/Linux) ou **%USERPROFILE%\cargo\bin** (Windows) est présent dans votre **PATH**.



2.2 LangQuest



Figure 4 - LangQuest - vocabulary quiz tool

LangQuest est un outil de quiz de vocabulaire en terminal utilisé pour pratiquer la terminologie et les concepts de programmation. Il est distribué en tant que binaire sur [LangQuest sur Github](#).

2.2.1 Installation



```
1 curl -fsSL https://raw.githubusercontent.com/tschinz/langquest/refs/heads/main/scripts/install_latest_release.sh | sh
```

LangQuest est installé sous **\$HOME/.local/bin**.



```
1 irm https://raw.githubusercontent.com/tschinz/langquest/refs/heads/main/scripts/install_latest_release.ps1 | iex
```

LangQuest est installé sous **%LOCALAPPDATA%\Programs\lq\bin\lq.exe**.

Vérifiez que la commande suivante affiche un numéro de version sans erreur:

```
1 lq -k
```

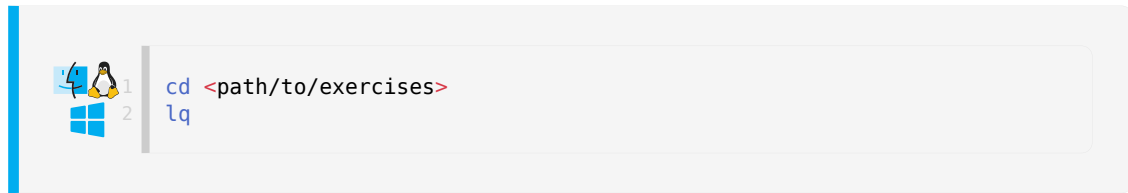
Contrôlez que les clés correspondent aux suivantes :



```
1 progress_key_sha256:
869e730b3d6be463c5474a41f802711943e330827a7c4ddd6bcd62dc9f4abc3a
2 attest_key_sha256:
af30b6a2512c11e8015da1522b55d61926f96591c3193a63f2d1b64f6f53b5b7
3 solution_key_sha256:
1d142c246028f02e928682c88b3280412d4ff82b0cbc275db68a236947c89283
```

2.2.2 Utilisation rapide

Pour l'exécuter, placez-vous dans un dossier contenant des exercices LangQuest puis lancez :



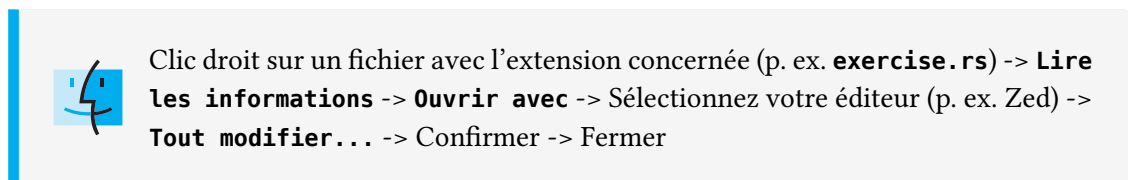
Le programme détecte automatiquement la structure et le contenu des exercices.

2.2.3 Associer les fichiers à l'éditeur

LangQuest ouvre les exercices dans l'éditeur défini par défaut pour :

- **Rust:** *.rs
- **Python:** *.py
- **Go:** *.go
- **C++:** *.c, *.h, *.cpp, *.hpp
- **RISC-V:** *.s, *.asm
- **Markdown:** *.md

Pour simplifier l'usage, associez les extensions utilisées dans LangQuest à l'éditeur de votre choix :

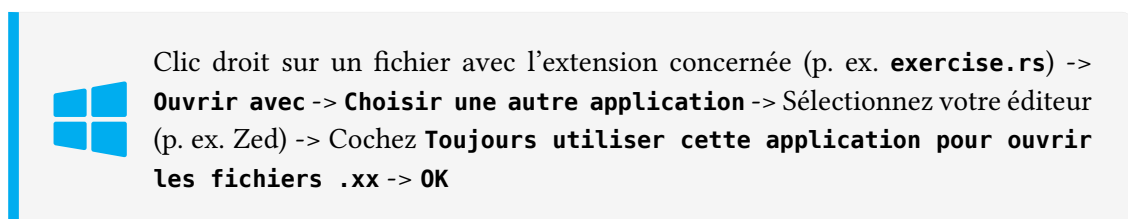


Clic droit sur un fichier avec l'extension concernée (p. ex. **exercice.rs**) -> **Lire les informations** -> **Ouvrir avec** -> Sélectionnez votre éditeur (p. ex. Zed) -> **Tout modifier...** -> Confirmer -> Fermer



Dépend du système

Ex. interface Ubuntu : **Paramètres** -> **Applications par défaut** -> **Définir les applications par type de fichier** -> Rechercher les extensions (p. ex. **.rs**) -> Sélectionner votre éditeur (p. ex. VSCode ou Zed) -> Fermer



Clic droit sur un fichier avec l'extension concernée (p. ex. **exercice.rs**) -> **Ouvrir avec** -> **Choisir une autre application** -> Sélectionnez votre éditeur (p. ex. Zed) -> Cochez **Toujours utiliser cette application pour ouvrir les fichiers .xx** -> OK



2.3 Éditeur de code Zed



Figure 5 - Zed - high-performance code editor

Zed est un éditeur de code haute performance, collaboratif, développé en Rust. C'est l'éditeur principal utilisé dans ce guide.

2.3.1 Installation

Visitez zed.dev et téléchargez l'installateur pour votre plateforme.



Téléchargez le **.dmg**, ouvrez-le puis glissez *Zed* dans le dossier *Applications*.

Installez l'outil CLI de Zed depuis Zed : Appuyez sur **cmd-shift-p** puis tapez **cli:install cli binary**



Ouvrez un terminal et exécutez le script d'installation officiel :

```
curl https://zed.dev/install.sh | sh
```



Téléchargez et exécutez l'installateur **.exe**.

Ajoutez **zed.exe** à votre **PATH** si ce n'est pas fait automatiquement.

2.3.2 Vérification



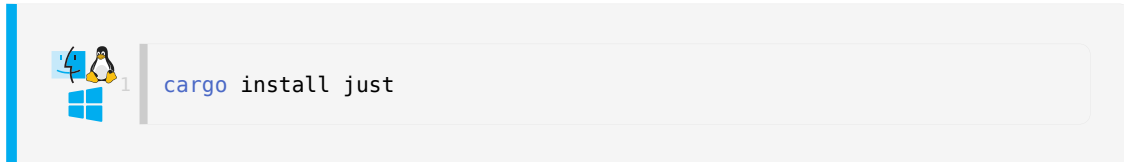
Ouvrez Zed et vérifiez que l'écran d'accueil apparaît. Connectez-vous avec votre compte GitHub pour activer les fonctionnalités **Artificial Intelligence (AI)**.



2.4 Just - Exécuteur de commandes

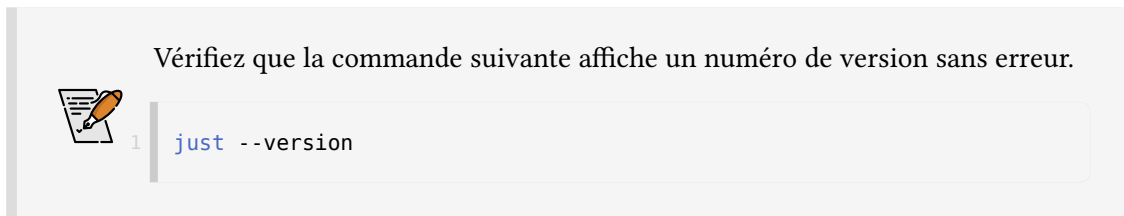
just est un exécuteur de commandes pratique (similaire à **make**) qui lit un **justfile** dans votre projet. Il est distribué comme crate Rust binaire et s'installe via **cargo**.

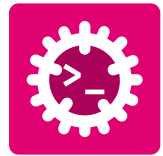
2.4.1 Installation



La première installation peut prendre quelques minutes car le crate et ses dépendances sont téléchargées puis compilées.

2.4.2 Vérification





3 | Guide LangQuest

3.1 Lancer LangQuest

LangQuest n'impose pas d'emplacement particulier pour les exercices, mais l'arborescence doit respecter une structure précise :

```
my-exercises/
├── lq.toml                ← auto-created by lq on first run
├── 01-basics/             ← module (prefixed with NN-)
│   ├── 01-hello-world/   ← exercise (prefixed with NN-)
│   │   ├── 01-theory.md
│   │   ├── 02-task.md
│   │   ├── main.rs
│   │   └── solution/
│   │       ├── main.rs
│   │       └── solution.md
│   └── 02-variables/
│       └── ...
└── 02-control-flow/
    └── ...
```

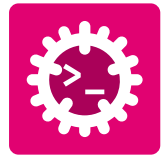
Enregistrez les exercices de votre cours dans un dossier dédié.

Dans un terminal, placez-vous dans le dossier contenant les exercices puis lancez **lq** pour démarrer LangQuest :

```
cd <path/to/exercises>
lq
```



Il est recommandé de l'exécuter dans un terminal ouvert dans **Zed** pour ouvrir directement les fichiers d'exercice dans le même éditeur. Les captures de ce guide proviennent de Zed.



3.2 Page d'ensemble

Ouvrez dans **Zed** le dossier contenant les exercices :

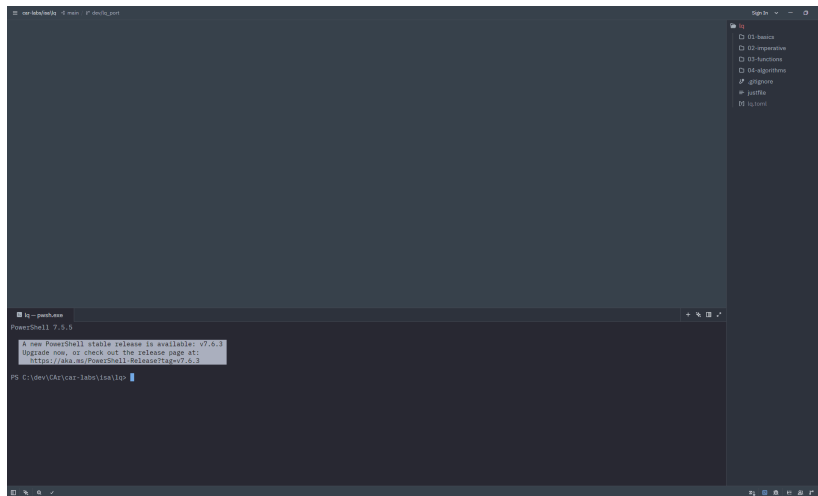


Figure 6 - Open exercises in Zed

Ouvrez le terminal via **View > Terminal Panel**, tapez **lq** puis appuyez sur **Enter** :

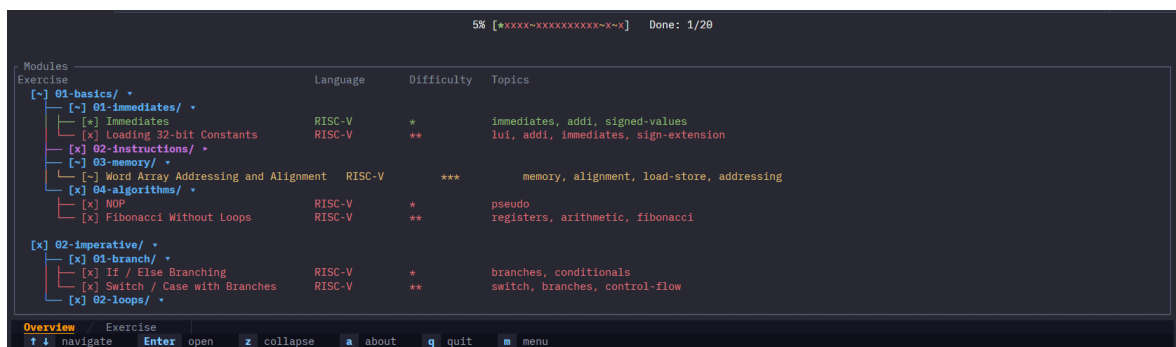
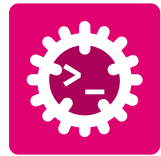


Figure 7 - **lq** in Zed



Si le programme ne démarre pas, consultez le terminal. La plupart des erreurs viennent de l'exécution de **lq** hors d'un dossier d'exercices, ou d'une structure invalide.



L'interface du programme est la suivante :

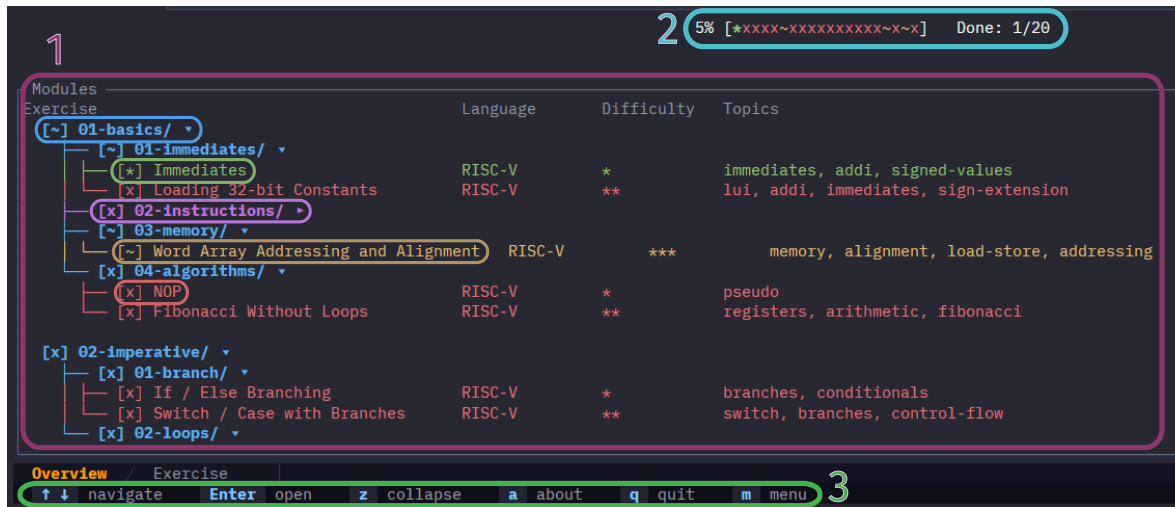


Figure 8 - lq overview page

1. Liste des exercices

Les exercices détectés sont affichés dans l'ordre. Parcourez-les avec les touches `↓ ↑` et sélectionnez-en un avec **Enter**.

- Les lignes **bleues** indiquent les modules/groupes d'exercices. Appuyez sur ***z*** pour replier/déplier les modules.
- La ligne **violette** représente la position du curseur.
- La ligne **verte** indique les exercices entièrement complétés.
- La ligne **jaune** indique les exercices partiellement complétés.
- La ligne **rouge** indique les exercices non complétés.

Pour chaque exercice, la vue affiche le langage (Rust, Go, C++, Python, assembleur RISC-V et Markdown), sa difficulté relative dans le module et les thèmes abordés.

3.2.1 Résumé des raccourcis

- `↓ ↑` : naviguer dans la liste des exercices
- **Enter** : déplier un module / sélectionner un exercice
- ***z*** : replier/déplier tous les modules dans la liste
- ***a*** : afficher la page à propos
- ***q*** / ***Esc*** : quitter le programme
- ***m*** : afficher/masquer le menu des raccourcis

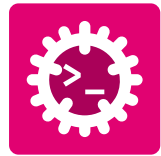
2. Progression

La barre de progression affiche le nombre d'exercices complétés. Elle est mise à jour selon l'état de chaque exercice :

- ***** : exercice complètement valide
- **~** : exigences partiellement satisfaites
- **x** : exigences non satisfaites

3. Menu

Liste des raccourcis clavier disponibles pour l'écran courant. Appuyez sur ***m*** pour masquer/afficher le menu.



3.3 Page des exercices

Les vues se parcourent avec les touches ← →.

3.3.1 Théorie

En entrant dans un exercice, l'onglet théorie s'affiche. Il contient une brève description et des rappels des concepts nécessaires pour résoudre l'exercice.

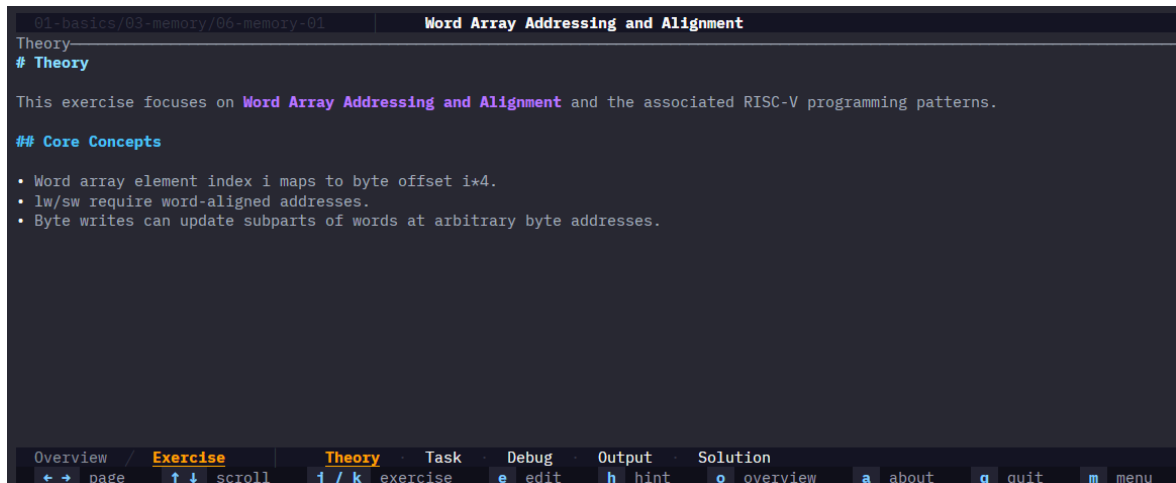


Figure 9 - Theory tab

3.3.2 Tâches

La vue des tâches contient la liste des tâches à réaliser pour l'exercice, des extraits de code, des consignes ...

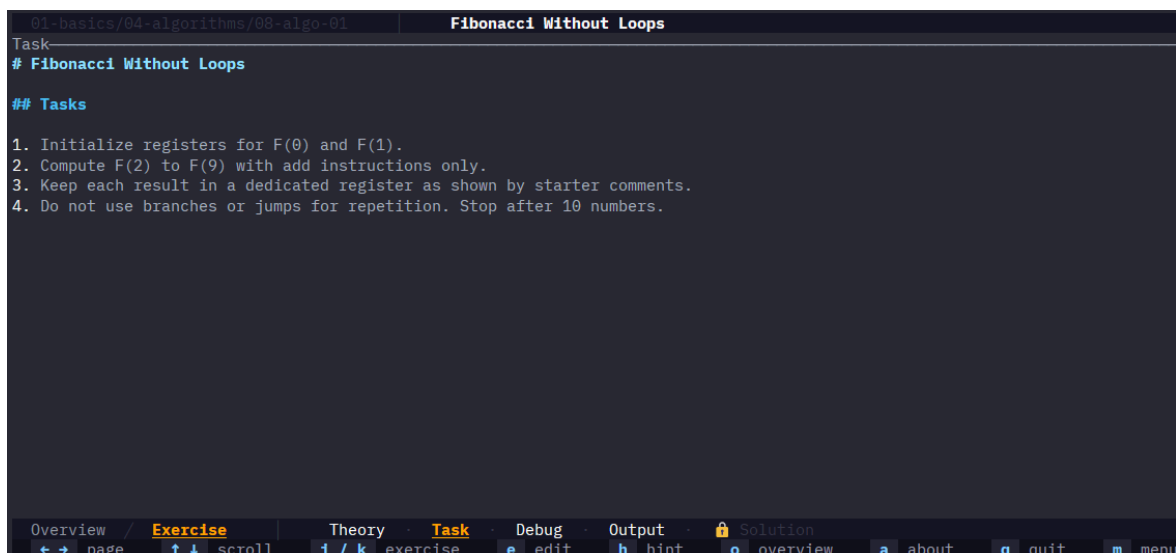


Figure 10 - Tasks tab

3.3.3 Débogage

La fenêtre de debug affiche les sorties **stdout** et **stderr** du programme testé. *Cette fenêtre n'est pas utile pour RISC-V et Markdown.*

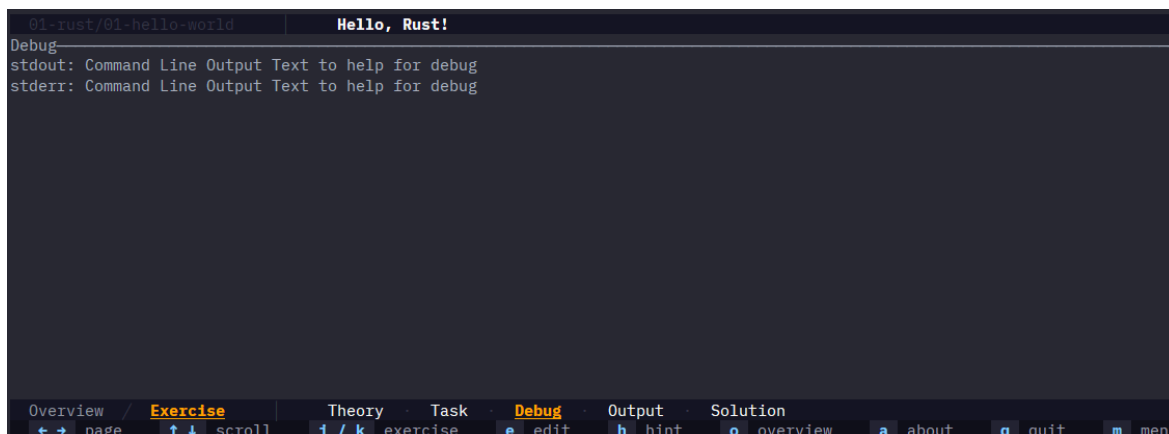


Figure 11 - Debug tab

3.3.4 Résultats

La fenêtre de sortie affiche les résultats des tests. Les tests sont définis directement dans le code de l'exercice, consultez-les en cas d'échec pour mieux comprendre.

Elle montre le total de tests réussis ainsi que ceux en échec, avec un format spécifique au langage. Le seuil à droite est le pourcentage minimal de tests à réussir pour que l'exercice soit considéré **partiellement complété** ou **complètement complété**.

Ce seuil est surtout utile pour les exercices Markdown, selon leur méthode de test.

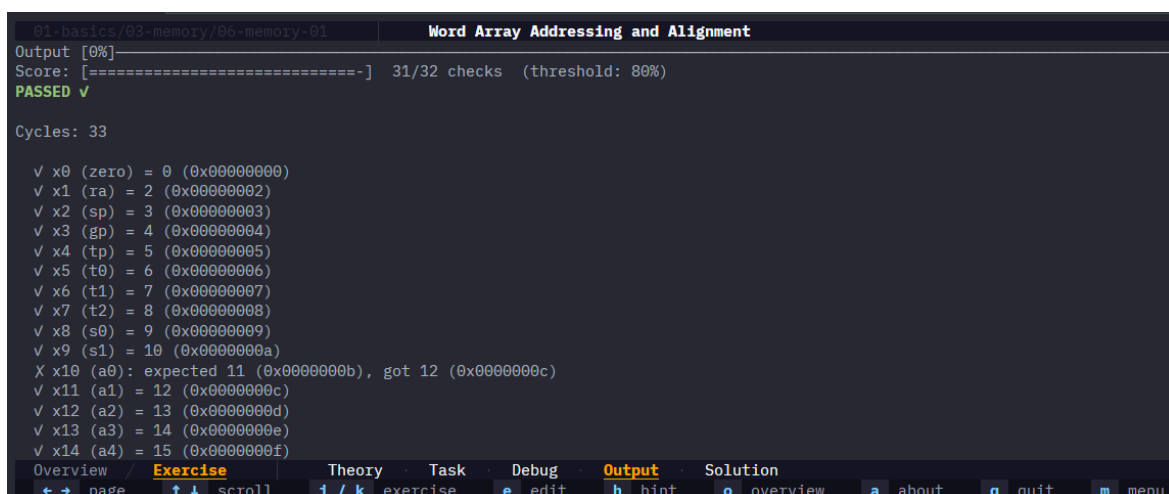


Figure 12 - Output tab

Appuyez sur ***e*** pour ouvrir l'exercice dans l'éditeur par défaut configuré à Chapitre 2.2.3. S'il s'agit du même éditeur que celui utilisé pour lancer **lq** (p. ex. Zed), le fichier s'ouvre dans un nouvel onglet de la même fenêtre.

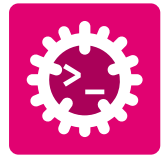
3.3.5 Indices et solution

Par défaut, la solution est verrouillée comme indiqué par l'icône de cadenas  **Solution**

Elle se débloquent sous deux conditions :

- Le seuil de complétion de l'exercice est atteint.
- Tous les indices sont débloqués.

Pour débloquent les indices, appuyez sur ***h*** afin d'afficher la vue suivante dans l'onglet **Output** :



```

01-basics/04-algorithms/08-algo-01 | Fibonacci Without Loops
Output
Set the two seed values first: 0 and 1.
[HINT 2/2]
For each next term, add the previous two registers.

⚠ The solution will be unlocked. Are you really sure?
Press 'h' again to confirm, or any other key to cancel.

Overview / Exercise Theory Task Debug Output Solution
⬅ ➡ page ⬆ ⬇ scroll j / k exercise e edit h hint o overview a about q quit m menu
    
```

Figure 13 - Output tab

Une fois tous les indices débloqués, appuyer à nouveau sur "**h**" débloque la solution, qui affiche la vue suivante :

```

01-basics/02-instructions/03-instruction-01 | Arithmetic Expression (Positive Values)
Solution
— Reference Solution —

# EXPECT_REG: t0 1
# EXPECT_REG: t1 2
# EXPECT_REG: t2 5
# EXPECT_REG: s0 -2

_start:
# a = b + c - d;
# t0 = b, t1 = c, t2 = d. s0 = d

# b = 1, c = 2, d = 5
addi t0, zero, 1
addi t1, zero, 2
addi t2, zero, 5
# t = b + c
add t3, t0, t1
# a = t - c
sub s0, t3, t2

Explanation
## Explanation

You practice expression lowering into primitive arithmetic instructions with explicit temporaries.

Read solution/main.asm together with the hints, then compare with main.asm to understand each implementation choice.

Overview / Exercise Theory Task Debug Output Solution
⬅ ➡ page ⬆ ⬇ scroll j / k exercise e edit h hint o overview a about q quit m menu
    
```

Figure 14 - Solution tab

3.3.6 Résumé des raccourcis

- **← →** : naviguer entre les vues de l'exercice
- **↓ ↑** : défiler dans les vues
- ***j*** / ***k*** : exercice précédent/suivant
- ***e*** : ouvrir l'exercice dans l'éditeur par défaut
- ***h*** : afficher les indices + débloquer la solution
- ***o*** : revenir à la page d'ensemble - Chapitre 3.2
- ***a*** : afficher la page à propos
- ***q*** / ***Esc*** : quitter le programme
- ***m*** : afficher/masquer le menu des raccourcis



Problème	Source	Étapes de résolution
Le programme se lance mais se ferme immédiatement	Dossier de lancement incorrect ou arborescence d'exercices invalide	1. Consultez la sortie du terminal. La plupart du temps, il indique qu'aucun exercice n'a été trouvé. 2. Placez-vous dans le dossier racine contenant les modules d'exercices. 3. Vérifiez que la structure des dossiers correspond au modèle attendu NN-module/NN-exercice. 4. Relancez lq depuis ce dossier racine. 5. Ouvrez un ticket sur https://github.com/tschinz/langquest/issues.
La touche e n'ouvre pas les fichiers dans l'éditeur attendu	Les associations d'extensions sont absentes ou pointent vers une autre application	1. Reconfigurez les applications par défaut pour .rs, .py, .go, .cpp, .md, etc. 2. Fermez et rouvrez l'éditeur. 3. Relancez lq.
La solution reste verrouillée	Seuil de complétion non atteint ou indices pas tous débloqués	1. Corrigez votre code pour réussir les tests jusqu'à atteindre le seuil. 2. Appuyez sur h jusqu'à débloquer tous les indices. 3. Appuyez encore sur h pour débloquer l'onglet solution.
L'interface se bloque ou ne répond plus	Actuellement, lq travaille sur un seul thread. Lorsqu'un exercice est ouvert ou le fichier d'exercice sauvegardé, une compilation + exécution dudit fichier est réalisée. Si l'une de ces étapes prend du temps, l'interface peut sembler bloquée. Par exemple, une boucle infinie dans le code de l'exercice peut provoquer un blocage.	1. Vérifiez que le code de l'exercice ne contient pas de boucle infinie. 2. Tentez une compilation en dehors de lq. 3. Contrôlez si une version plus récente de LangQuest est disponible.



Glossary

AI – Artificial Intelligence: Field of computer science focused on creating systems capable of performing tasks that typically require human intelligence. [9](#)